

Multiliquid v2

Security Review

Cantina Managed review by:
Desmond Ho, Lead Security Researcher
Rvierdiiev, Security Researcher

November 19, 2025

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 Medium Risk . . . 4
3.1.1 Fee is overcharged for RWA to RWA swap exactOut case . . . 4
3.1.2 Redemption fee is overcharged for stablecoin to RWA swaps . . . 4
3.1.3 Protocol fee calculation is incorrect for exact out RWA to RWA swaps . . . 4
3.2 Low Risk . . . 5
3.2.1 Protocol fee is calculated based on stablecoin amount instead of usd amount . . . 5
3.2.2 Protocol and redemption fee are not converted to stablecoin in case of rwa to rwa swaps . . . 6
3.2.3 Rounding issues may cause dust wei to be unaccounted for . . . 6
3.3 Informational . . . 7
3.3.1 Deprecated `blacklist` can be renamed for clarity on its deprecation . . . 7
3.3.2 Refactor slippage protection checks into the internal `_calculateStablecoinToStablecoin` function . . . 7
3.3.3 Redundant `depositStablecoin()` function . . . 7
3.3.4 Incorrect comment . . . 8
3.3.5 Unused `RWA_DELEGATE_ROLE` . . . 8
3.3.6 Deprecated blacklist functionality . . . 8
3.3.7 Sanity check identical RWA ID swaps . . . 8
3.3.8 Minor wei differences due to precision differences for exact in vs out specifications . . . 8
3.3.9 Consider using `burnFrom()` instead of `burn(address, uint256)` interface . . . 10

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Nov 5th to Nov 11th the Cantina team conducted a review of Multiliquid on commit hash 006e642f. The team identified a total of **15** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	3	3	0
Low Risk	3	3	0
Gas Optimizations	0	0	0
Informational	9	8	1
Total	**15**	**14**	**1**

2.1 Scope

The security review had the following components in scope for Multiliquid on commit hash 006e642f:

```
src/v2
├── BalanceSheetStablecoinDelegate.sol
├── MintBurnStablecoinDelegate.sol
├── MultiliquidSwap.sol
├── PriceAdapters
│   └── ThirdPartySetterPriceAdapter.sol
├── ProtocolStablecoinDelegates
│   ├── BraleStablecoinDelegate.sol
│   ├── GenericMintBurnStablecoinDelegate.sol
│   └── UniformLabsDelegate.sol
├── stablecoin
│   └── LiquidTreasury.sol
└── StablecoinDelegateBase.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Fee is overcharged for RWA to RWA swap exactOut case

Severity: Medium Risk

Context: MultiliquidSwap.sol#L1377-L1380

Description: For the RWA to RWA swap in case of `exactOut`, the redemption fee is currently calculated as follows:

```
outputs.redemptionFeeAmt = _fromWad(
    (rwaInUSDWad.divWadUp(WAD - redemptionFeeRateIn) - rwaInUSDWad)
        + (rwaOutUSDWad - rwaOutAmtWad.mulWad(rwaOutPrice)),
    params.stablecoinDecimals
);
```

In this formula, the redemption fee on the input is effectively applied twice, since `rwaInUSDWad` already includes the fee. As a result, the user is charged a higher redemption fee than intended.

Recommendation: Use the following corrected formula to calculate the total redemption fee:

```
outputs.redemptionFeeAmt = _fromWad(
    rwaInUSDWad - rwaOutAmtWad.mulWad(rwaOutPrice),
    params.stablecoinDecimals
);
```

Multiliquid: Fixed in commit 1ce7b929.

Cantina Managed: Fix verified.

3.1.2 Redemption fee is overcharged for stablecoin to RWA swaps

Severity: Medium Risk

Context: MultiliquidSwap.sol#L1276

Description: `swapIntoRWA()` function charge the redemption fee based on the entire stablecoin amount. However, not the whole stablecoin amount is actually swapped into the RWA - a portion is first deducted as a protocol fee. Therefore, only `stablecoinAmount - protocolFee` is effectively swapped and should be used to calculate the redemption fee.

As a result, users are currently overcharged. The correct approach is already used in other functions such as:

- `swapIntoRWAExactOut()`.
- `swapRWAToRWA()`.
- `swapIntoStablecoin()`.
- `swapStablecoinToStablecoin()`.

Recommendation: Align fee calculation across all swap functions, ensuring that only `stablecoinAmount - protocolFee` is used when calculating the redemption fee.

Multiliquid: Fixed in commit 5d00b225.

Cantina Managed: Fix verified.

3.1.3 Protocol fee calculation is incorrect for exact out RWA to RWA swaps

Severity: Medium Risk

Context: MultiliquidSwap.sol#L1363-L1367

Description: There is a discrepancy in the calculation of the protocol fee for the exact out case, where the amount calculated is less than expected.

Proof of Concept: Adding console logs to the function:

```
console.log("protocolFeeWad", protocolFeeWad);
console.log("rwaInUSDWad", rwaInUSDWad);
console.log("### FOWARD CALC ###");
uint256 protocolFeeWadReversed =
↪  totalRwaInUSDWad.mulWadUp(_calculateMultiliquidFee(rwaInUSDWad));
console.log(
    "protocolFeeWadReversed",
    protocolFeeWadReversed
);
console.log("usdAfterProtocolFeeWad", totalRwaInUSDWad - protocolFeeWadReversed);
```

Current behaviour:

```
[PASS] test_swapRWAToRWA_exactOut_with_higher_redemptionFees() (gas: 508957)
Logs:
  protocolFeeWad 265144819575263251
  rwaInUSDWad 53028963915052650186
  ### FOWARD CALC ###
  protocolFeeWadReversed 266470543673139568
  usdAfterProtocolFeeWad 53027638190954773869
```

With the change:

```
[FAIL: Higher redemption fees should increase input required: 46343861844048634640 !=
↪  46342703247502533424] test_swapRWAToRWA_exactOut_with_higher_redemptionFees() (gas:
↪  520351)
Logs:
  protocolFeeWad 266477205603279650
  rwaInUSDWad 53028963915052650186
  ### FOWARD CALC ###
  protocolFeeWadReversed 266477205603279650
  usdAfterProtocolFeeWad 53028963915052650186
```

Recommendation:

```
// Step 3: Reverse protocol fee to find total rwaIn USD needed
uint256 totalRwaInUSDWad = rwaInUSDWad.divWadUp(WAD -
↪  _calculateMultiliquidFee(rwaInUSDWad));

// Step 4: Calculate protocol fee from difference
uint256 protocolFeeWad = totalRwaInUSDWad - rwaInUSDWad;
```

Multiliquid: Fixed in commit 802ba0f0.

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 Protocol fee is calculated based on stablecoin amount instead of usd amount

Severity: Low Risk

Context: MultiliquidSwap.sol#L1269, MultiliquidSwap.sol#L1305, MultiliquidSwap.sol#L1441, MultiliquidSwap.sol#L1477, MultiliquidSwap.sol#L1542, MultiliquidSwap.sol#L1565-L1566

Description: When a user performs a swap, the fee is determined according to the defined fee tiers.

```
function _calculateMultiliquidFee(uint256 stableAmt) internal view returns (uint256
↪  feeBps) {
    if (feeTiers.length == 0) revert InvalidFeeTiers();
    uint256 length = feeTiers.length;
    for (uint256 i = 0; i < length;) {
        if (stableAmt <= feeTiers[i].maxVolume) {
            return feeTiers[i].fee;
        }
```

```
        unchecked {
            ++i;
        }
    }
    // If the stableAmt is greater than the max volume of the last fee tier, use the last
    ↪  fee tier
    return feeTiers[feeTiers.length - 1].fee;
}
```

`FeeTier.maxVolume` is denominated in USD, which means that all compared amounts should be converted to USD value before comparison. However, for stablecoins, this conversion is skipped - the code directly uses the stablecoin amount. As a result, the protocol may miscalculate the fee for stablecoin swaps.

Recommendation: Convert the stablecoin amount to its USD equivalent before calculating the applicable fee tier.

Multiliquid: Fixed in commit 21d2ea00.

Cantina Managed: Fix verified.

3.2.2 Protocol and redemption fee are not converted to stablecoin in case of rwa to rwa swaps

Severity: Low Risk

Context: MultiliquidSwap.sol#L1375-L1381, MultiliquidSwap.sol#L1408-L1412

Description: When swapping from RWA to RWA, the calculations inside the `_calculateRWAToRWA()` function are performed in USD, including both the protocol and redemption fees. However, these fees are later passed to the stablecoin delegate without converting them from USD to the stablecoin denomination, resulting in incorrect fee amounts being used.

Recommendation: Convert the protocol and redemption fees from USD to stablecoin value inside the `_calculateRWAToRWA()` function before passing them to the stablecoin delegate.

Multiliquid: Fixed in commit 94eb1c9b.

Cantina Managed: Fix verified.

3.2.3 Rounding issues may cause dust wei to be unaccounted for

Severity: Low Risk

Context: MultiliquidSwap.sol#L1548-L1550

Description: Because of rounding, it's possible that `stablecoinOutUSDWad + protocolFeeWad + acceptanceFee`

Proof of Concept: Add this invariant:

```
if (stablecoinOutUSDWad + protocolFeeWad + acceptanceFeeWad + redemptionFeeWad !=
↪  stablecoinInUSDWad) revert("INVARIANT FAILED");
```

Add this fuzz test in `StableToStableBalanceSheetTest`:

```
/// forge-config: default.fuzz.runs = 100_000
function test_fuzzExactOutRounding(uint256 exactAmountOut) public {
    exactAmountOut = bound(exactAmountOut, 1e6, 1_000_000e6);
    uint256 maxAmountIn = type(uint256).max;

    multiliquidSwap.calculateStablecoinToStablecoin(
        true,
        false,
        stablecoinID1,
        maxAmountIn,
        stablecoinID3,
        exactAmountOut
    );
}
```

There should have a counter-example produced from running the test.

Recommendation: Consider calculating `protocolFeeWad` as the remainder (difference), because either `acceptanceFeeWad` or `redemptionFeeWad` is zero for the stablecoin swap.

```
protocolFeeWad = stablecoinInUSDWad - stablecoinOutUSDWad - redemptionFeeWad -
↪  acceptanceFeeWad;
```

Multiliquid: Fixed in commit 6bf2c7d4.

Cantina Managed: Fix verified.

3.3 Informational

3.3.1 Deprecated `blacklist` can be renamed for clarity on its deprecation

Severity: Informational

Context: MultiliquidSwap.sol#L169

Description/Recommendation: Can perhaps be renamed to `DEPRECATED_MAPPING` for clarity.

Multiliquid: Fixed in commit fbe6bd19.

Cantina Managed: Fix verified.

3.3.2 Refactor slippage protection checks into the internal `_calculateStablecoinToStablecoin` function

Severity: Informational

Context: MultiliquidSwap.sol#L976

Description/Recommendation: The slippage protection checks that exist in `swapStablecoinToStablecoin` are absent in the corresponding calculation function.

```
if (inputs.exactOut) {
    // exactOut: check that stablecoinIn required doesn't exceed maximum user is willing
    ↪  to spend
    if (outputs.stablecoinInAmount > inputs.stablecoinInAmount) {
        revert InsufficientStablecoinInput();
    }
} else {
    // exactIn: check that stablecoinOut received meets minimum user expects
    if (outputs.stablecoinOutAmount < inputs.stablecoinOutAmount) {
        revert InsufficientStablecoinOutput();
    }
}
```

Consider refactoring these checks into the internal `_calculateStablecoinToStablecoin` function.

Multiliquid: Fixed in commit fbe6bd19.

Cantina Managed: Fix verified.

3.3.3 Redundant `depositStablecoin()` function

Severity: Informational

Context: StablecoinDelegateBase.sol#L202-L213

Description: The delegate is not expected to be holding any assets, as the RWA and stablecoin assets should be held by the relevant custody wallets which can't be set to `address(this)`.

Recommendation: Remove `depositStablecoin()`, and consider generalising the `withdrawStablecoin()` function to withdraw any ERC20 that's accidentally sent to this contract.

Multiliquid: Fixed in commit f478b386.

Cantina Managed: Fix verified.

3.3.4 Incorrect comment

Severity: Informational

Context: LiquidTreasury.sol#L199

Description: Incorrect comment as token approval is not required for direct burns.

Recommendation: Remove the referenced comment.

Multiliquid: Fixed in commit f478b386.

Cantina Managed: Fix verified.

3.3.5 Unused `RWA_DELEGATE_ROLE`

Severity: Informational

Context: MultiliquidSwap.sol#L45

Description: The `RWA_DELEGATE_ROLE` is defined but unused.

Recommendation: Remove the referenced role.

Multiliquid: Fixed in commit f478b386.

Cantina Managed: Fix verified.

3.3.6 Deprecated blacklist functionality

Severity: Informational

Context: RWADelegate.sol#L138-L141

Description: The blacklist functionality has been deprecated in V2 and has been modified to whitelist in the delegates themselves.

Recommendation: Remove the referenced function. Since `RWADelegate` is inherited by `ULTRADelegate`, if it needs the whitelist functionality, then the `stablecoinWhitelist` mapping and setter function should be added.

Multiliquid: Fixed in commit f478b386.

Cantina Managed: Fix verified.

3.3.7 Sanity check identical RWA ID swaps

Severity: Informational

Context: MultiliquidSwap.sol#L493

Description/Recommendation: Consier sanity checking that `rwaInID != rwaOutID`.

Multiliquid: Fixed in commit 6f237633.

Cantina Managed: Fix verified.

3.3.8 Minor wei differences due to precision differences for exact in vs out specifications

Severity: Informational

Context: MultiliquidSwap.sol#L976

Description: There are some minor differences in the `in` & `out` amounts calculated due to truncation when specifying an amount with a stablecoin of higher decimals.

In the examples below for a DAI <> USDC swap with the existing USDC delegate, it would be slightly more advantageous to specify the exact DAI output amount for USDC -> DAI (same USDC in, but get a little more out, `100e18` vs `99.999e18`), and the exact USDC output amount for the DAI -> USDC swap (less DAI requested).

```
function test_feeEquivalence_daiToUsdc() public {
    uint256 exactInDaiAmount = 100e18 + 123456789;
    vm.startPrank(multiliquidAdmin);
    multiliquidSwapV2.setStablecoinAcceptanceForSwap(
        daiID,
        dai,
        true
    );
    multiliquidSwapV2.setStablecoinUSDValue(daiID, 1e18);
    vm.stopPrank();

    vm.prank(usdcDelegateIssuerAdmin);
    // set acceptance fee amount for DAI -> USDC
    usdcDelegateV2.setStablecoinAcceptanceFee(dai, 1e18 / 100_000); // 0.001%

    IMultiliquidSwapV2.StablecoinToStablecoinReturnParams memory exactInParams =
    ↪  multiliquidSwapV2.calculateStablecoinToStablecoin(
        false, // exactOut
        true, // useDelegateForStablecoinOut
        daiID,
        exactInDaiAmount,
        usdcID,
        0
    );

    IMultiliquidSwapV2.StablecoinToStablecoinReturnParams memory exactOutParams =
    ↪  multiliquidSwapV2.calculateStablecoinToStablecoin(
        true, // exactOut
        true, // useDelegateForStablecoinOut
        daiID,
        type(uint256).max,
        usdcID,
        exactInParams.stablecoinOutAmount
    );

    console.log("exactInParams.stablecoinInAmount", exactInParams.stablecoinInAmount);
    console.log("exactOutParams.stablecoinInAmount", exactOutParams.stablecoinInAmount);
    console.log("exactInParams.stablecoinOutAmount", exactInParams.stablecoinOutAmount);
    console.log("exactOutParams.stablecoinOutAmount",
    ↪  exactOutParams.stablecoinOutAmount);
    console.log("exactInParams.protocolFeeAmt", exactInParams.protocolFeeAmt);
    console.log("exactOutParams.protocolFeeAmt", exactOutParams.protocolFeeAmt);
    console.log("exactInParams.acceptanceFeeAmt", exactInParams.acceptanceFeeAmt);
    console.log("exactOutParams.acceptanceFeeAmt", exactOutParams.acceptanceFeeAmt);
    console.log("exactInParams.redemptionFeeAmt", exactInParams.redemptionFeeAmt);
    console.log("exactOutParams.redemptionFeeAmt", exactOutParams.redemptionFeeAmt);

    // exactInParams should be == exactOutParams
    assertEq(exactOutParams.stablecoinInAmount, exactInParams.stablecoinInAmount);
    assertEq(exactOutParams.stablecoinOutAmount, exactInParams.stablecoinOutAmount);
    assertEq(exactOutParams.protocolFeeAmt, exactInParams.protocolFeeAmt);
    assertEq(exactOutParams.acceptanceFeeAmt, exactInParams.acceptanceFeeAmt);
    assertEq(exactOutParams.redemptionFeeAmt, exactInParams.redemptionFeeAmt);
}
```

will get `exactOutParams.stablecoinInAmount` as `100e18`.

```
function test_fuzzFeeEquivalence_usdcToDaiStartWithExactOut(uint256 exactOutDaiAmount)
↪  public {
    exactOutDaiAmount = bound(exactOutDaiAmount, 100e18, 1_000_000e18);
    vm.startPrank(multiliquidAdmin);
    multiliquidSwapV2.setStablecoinAcceptanceForSwap(
        daiID,
        dai,
        true
```

```
    );
    multiliquidSwapV2.setStablecoinUSDValue(daiID, 1e18);
    vm.stopPrank();

    // set redemption fee amount for USDC -> DAI
    vm.startPrank(usdcDelegateIssuerAdmin);
    usdcDelegateV2.setStablecoinRedemptionFee(dai, 1e18 / 20_000); // 0.005%

    // set acceptance fee amount for DAI -> USDC
    usdcDelegateV2.setStablecoinAcceptanceFee(dai, 1e18 / 100_000); // 0.001%
    vm.stopPrank();

    // USDC -> DAI, want to use USDC delegate, so useDelegateForStablecoinOut is false
    IMultiliquidSwapV2.StablecoinToStablecoinReturnParams memory exactOutParams =
    ↪  multiliquidSwapV2.calculateStablecoinToStablecoin(
        true, // exactOut
        false, // useDelegateForStablecoinOut
        usdcID,
        type(uint256).max,
        daiID,
        exactOutDaiAmount
    );

    IMultiliquidSwapV2.StablecoinToStablecoinReturnParams memory exactInParams =
    ↪  multiliquidSwapV2.calculateStablecoinToStablecoin(
        false, // exactOut
        false, // useDelegateForStablecoinOut
        usdcID,
        exactOutParams.stablecoinInAmount,
        daiID,
        0
    );

    console.log("exactInParams.stablecoinInAmount", exactInParams.stablecoinInAmount);
    console.log("exactOutParams.stablecoinInAmount", exactOutParams.stablecoinInAmount);
    console.log("exactInParams.stablecoinOutAmount", exactInParams.stablecoinOutAmount);
    console.log("exactOutParams.stablecoinOutAmount",
    ↪  exactOutParams.stablecoinOutAmount);

    // exactInParams should be == exactOutParams
    assertEq(exactOutParams.stablecoinInAmount, exactInParams.stablecoinInAmount);
    assertEq(exactOutParams.stablecoinOutAmount, exactInParams.stablecoinOutAmount);
    assertEq(exactOutParams.protocolFeeAmt, exactInParams.protocolFeeAmt);
    assertEq(exactOutParams.acceptanceFeeAmt, exactInParams.acceptanceFeeAmt);
    assertEq(exactOutParams.redemptionFeeAmt, exactInParams.redemptionFeeAmt);
}
```

will have

```
exactInParams.stablecoinOutAmount 99999999634500000000
exactOutParams.stablecoinOutAmount 100000000000000000000
```

Recommendation: No action required as the overall system is generally permissioned, but these small differences is something to be aware of.

Multiliquid: Acknowledged.

Cantina Managed: Acknowledged.

3.3.9 Consider using `burnFrom()` instead of `burn(address, uint256)` interface

Severity: Informational

Context: MintBurnStablecoinDelegate.sol#L75, MintBurnStablecoinDelegate.sol#L142

Description: The `burn(address, uint256)` function is non-standard; the OpenZeppelin implementation that most burnable ERC20 tokens utilise does not have it. Instead, there is the `burnFrom()` method.

Recommendation: Replace the `burn()` function with `burnFrom()`.

Multiliquid: Fixed in commit dcffa8b3.

Cantina Managed: Fix verified.